

EdgeGuard AI

Working Document — Team Roles, Sensor Quotation & Microcontroller List
Reference Vehicle: Tata Signa 4825.TK (47.5t GVW, 38t Payload Tipper) | 5-Day Hackathon Build

1. Team Structure & Responsibilities

Member	Role	Core Responsibility	Key Deliverables
Member 1	Frontend Developer	Builds the 3-tier dashboard UI: live telemetry view, alerts panel, copilot chat interface.	Dashboard (web), alert cards, RUL display, copilot chat UI
Member 2	Full-Stack Developer (ML support + Phase 1 simulation)	Owns the FastAPI ingest service and MQTT listener; builds the Wokwi simulation feeding sensor data; supports Member 4 on data pipeline into the model.	FastAPI backend, MQTT broker integration, Wokwi simulation circuit, API contract with ML service
Member 3	Database Handler (ML support)	Designs and manages SQLite/Postgres schema for sensor readings, predictions, and SOP metadata; supports Member 4 with clean, queryable training data.	DB schema, ingestion-to-storage pipeline, data export for model training
Member 4	Machine Learning Engineer	Builds the synthetic degradation dataset, trains the XGBoost / LightGBM failure-prediction and RUL model, and integrates the RAG layer (Chroma/FAISS) for SOP retrieval.	Trained model, inference API, RAG pipeline, evaluation metrics
Member 5	Sensor Handler	Owns sensor selection, wiring logic in Wokwi, signal mapping to failure modes, and the sensor-to-MQTT topic schema.	Sensor list finalization, Wokwi sensor wiring, topic schema doc, sensor calibration notes

Shared interface contract (Day 1 priority): MQTT topic schema and the inference API request/response format must be locked before parallel build tracks start, so Members 1, 2, 3, 4, and 5 can work independently without blocking each other.

2. Sensor Quotation List

Indicative online prices in INR, sourced from Indian electronics retailers (Robu.in, ElectronicsComp, IndiaMART, RoboCraze) as of June 2026. Hackathon-grade modules are listed for Phase 1 simulation/prototyping; industrial-grade equivalents are listed separately for hydraulic and suspension pressure since these need pressure-rated transducers for any real Phase 2/3 deployment.

Sensor	Use / Failure Mode Tracked	Indicative Price (INR)
MPU6050 (Accelerometer + Gyroscope Module)	Vibration sensing at wheel hub / axle bearing — detects bearing damage and mechanical imbalance	Rs 100 - 250 / unit
DS18B20 (Waterproof Digital Temperature Probe)	Engine block / coolant outlet temperature — detects overheating and early bearing/engine stress	Rs 100 - 250 / unit
Analog Oil Pressure Sensor (0-10 Bar, e.g. G1/8 thread type)	Oil filter line / oil pan tap — detects leakage, pump failure, lubrication issues	Rs 600 - 1,500 / unit
Voltage Sensor Module (0-25V, resistor-divider type)	Battery voltage at terminals + alternator output — detects battery degradation, charging issues	Rs 50 - 150 / unit
Hydraulic Pressure Transducer (0-600 Bar, 4-20mA, prototype grade)	Tipper hydraulic ram/cylinder line — detects hydraulic wear, leakage, overload	Rs 2,000 - 4,500 / unit (entry-level industrial); Rs 6,000+ for branded/certified units
Suspension / Air Pressure Sensor (0-16 Bar, automotive-grade)	Air suspension bag or leaf-spring/shock mount per axle — detects suspension stress, structural wear	Rs 1,500 - 3,500 / unit
RPM — via J1939 CAN Bus (no separate sensor)	Pulled directly from ECU — idle runtime, workload, fuel inefficiency tracking	No sensor cost (CAN-to-USB/Wi-Fi dongle covers this, see below)
CAN-to-USB / CAN-to-Wi-Fi Dongle (J1939 compatible)	Taps the truck's J1939 CAN bus to read RPM and other ECU parameters without extra wiring	Rs 1,500 - 5,000 / unit depending on brand and Wi-Fi support

Estimated total (one full sensor set, Phase 1/2 prototype grade): roughly Rs 6,000 - 14,000 depending on hydraulic/suspension sensor grade chosen. Prices fluctuate by seller and bulk discount; treat as planning-stage estimates, not final quotes. Always verify current price and stock directly with the seller before purchase.

3. Microcontroller / Edge Device List

These are separate from the sensors above — this is the compute layer that reads sensor data and pushes it to MQTT (Phase 1 simulation uses Wokwi virtually; Phase 2 moves to a real board below).

Microcontroller / Board	Best Use Case	Indicative Price (INR)
ESP32 DevKitC (WROOM-32, 38-pin)	Primary recommendation for Phase 1/2 — built-in WiFi, dual-core, ideal for MQTT publishing directly from sensors. Used in the Wokwi simulation.	Rs 280 - 600 / unit
Arduino Uno R3 (or clone)	Good for early sensor testing/calibration on the bench, but needs an external WiFi/Ethernet shield for MQTT — ESP32 is generally a better fit for this project.	Rs 350 - 700 / unit
Raspberry Pi 4 (4GB variant)	Phase 2 edge device option if running the FastAPI service and ML inference directly on-device rather than cloud/laptop.	Rs 5,500 - 7,500 / unit (board only)
NVIDIA Jetson Nano	Only relevant if the multi-modal/vision (carryback detection) feature needs on-edge inference in Phase 2/3 — overkill for sensor telemetry alone.	Rs 12,000 - 16,000 / unit

Recommendation for this hackathon: stick to ESP32 as the single board reference in your pitch and BOM — it covers every sensor in this list over I2C/analog/digital pins and has native WiFi for MQTT, so there's no reason to mix in Arduino Uno or Raspberry Pi unless Phase 2 specifically needs on-device model inference.

4. Data Processing Workflow (Step-by-Step, by Owner)

End-to-end flow of a single sensor reading, from the truck to the dashboard alert — mapped to the team member who owns that step.

Step	Member	Role	Work Performed
1. Sensing	Member 5	Sensor Handler	Raw signal generated at sensor — temperature, vibration, oil pressure, RPM (CAN), battery voltage, hydraulic and suspension pressure. Defines sensor-to-topic mapping.
2. Simulation	Member 2	Full-Stack Dev (Phase 1 sim)	Wokwi circuit generates/streams the simulated sensor values in place of physical hardware, publishing them onto MQTT topics defined in Step 1.
3. Transport	Member 2	Full-Stack Dev	MQTT broker carries each sensor message from publisher (Wokwi) to subscriber (FastAPI backend). Handles topic routing.
4. Ingestion & Cleaning	Member 2	Full-Stack Dev	FastAPI background MQTT listener receives each message, validates format, removes spikes/invalid readings, and buffers data locally if connectivity drops.
5. Storage	Member 3	Database Handler	Cleaned readings are written to SQLite/Postgres with timestamp, sensor ID, and value — structured for fast retrieval by the ML service.
6. Feature Prep	Member 3 & 4	DB Handler + ML Engineer	Member 3 exposes query-ready data; Member 4 transforms raw readings into model features (rolling averages, rate of change, degradation trends).
7. Prediction	Member 4	ML Engineer	XGBoost/LightGBM model runs inference on prepared features — outputs failure probability and Remaining Useful Life (RUL) estimate.
8. Explanation (RAG)	Member 4	ML Engineer	If a failure risk is flagged, the RAG layer (Chroma/FAISS) retrieves the matching SOP/manual section to ground the copilot's explanation.
9. Presentation	Member 1	Frontend Developer	Dashboard displays the live telemetry, renders the alert with RUL, and shows the copilot's plain-language explanation to the user.

Read top to bottom for the live path of one sensor reading: Sense (5) → Simulate (2) → Transport (2) → Clean & Store (2, 3) → Predict (4) → Explain (4) → Display (1).

5. Machine Learning Model — Explained

Model name: Gradient-boosted decision trees — XGBoost (primary) or LightGBM (fallback if training speed becomes a constraint). Two models are trained from the same feature set: a **Failure Classifier** (will this component fail soon? yes/no, with probability) and a **RUL Regressor** (how many operating hours are left before failure?).

Why gradient-boosted trees, not deep learning

Sensor telemetry here is structured, tabular data (temperature, vibration, pressure, RPM readings over time) rather than images or raw audio. XGBoost/LightGBM consistently outperform neural networks on tabular data of this size, train in seconds to minutes on a laptop CPU with no GPU needed, and produce feature-importance scores that explain *why* a prediction was made — critical for the copilot's explanation layer and for judges who ask "how do you know it works."

How it works, step by step

- 1. Feature engineering:** raw sensor readings are converted into windowed features — rolling mean, rolling standard deviation, rate of change, and min/max over a trailing window (e.g. last 50 readings) for each sensor channel. This captures trends, not just single noisy readings.
- 2. Synthetic training data:** since no historical failure data exists yet, training data is generated by simulating the degradation cascade (temperature $\uparrow$ $\rightarrow$ vibration $\uparrow$ $\rightarrow$ oil pressure shift $\rightarrow$ failure) across many simulated truck-life cycles, with labeled failure points and known RUL at each timestep.
- 3. Training:** XGBoost builds an ensemble of shallow decision trees sequentially, where each new tree corrects the errors of the previous ones. The classifier learns the decision boundary between healthy and at-risk states; the regressor learns to estimate hours-to-failure.
- 4. Inference:** for every new batch of cleaned sensor readings, the FastAPI service computes the same features and calls both models, returning a failure probability and an RUL estimate in milliseconds.
- 5. Explainability:** feature importance (which sensor/feature drove the prediction) is passed to the RAG layer so the copilot can say, in plain language, e.g. "vibration has risen 40% over 6 hours, consistent with bearing wear — estimated 18 hours remaining."

Evaluation approach

Classifier: precision/recall/F1, with recall on the failure class prioritized (a missed failure is costlier than a false alarm).
Regressor: Mean Absolute Error (MAE) on predicted vs. actual RUL across held-out simulated truck-life cycles, reported in hours.

6. Database Details — Explained

Database choice: SQLite for the hackathon build (zero setup, file-based, runs anywhere on a laptop); structurally compatible with a drop-in move to local Postgres later with no schema redesign if concurrent write load grows.

Why SQLite over Postgres for this hackathon

Five people on laptops, no shared server, 5-day timeline. SQLite needs no install, no daemon, and no network config — one file that the FastAPI service, ML training scripts, and dashboard can all read directly. Postgres adds real value only once multiple services need concurrent write access at production scale, which is a Phase 2/3 concern, not a Phase 1 one.

Core tables

Table	Key Columns	Purpose
sensor_readings	id, truck_id, sensor_type, value, unit, timestamp	Stores every cleaned sensor reading as it arrives from the FastAPI ingest service — the raw time-series backbone.
predictions	id, truck_id, component, failure_probability, rul_hours, model_version, created_at	Stores each inference result from the ML model, linked back to the readings window that produced it.
alerts	id, truck_id, component, severity, message, sop_reference, created_at, acknowledged	Stores triggered alerts shown on the dashboard, including which SOP the copilot retrieved for the explanation.
sop_documents	id, title, component, content_chunk, embedding_ref	Stores the chunked SOP/manual text used by the RAG layer; embedding_ref points to the corresponding Chroma/FAISS vector.
trucks	id, model, gvw, payload_capacity, registration	Static reference table for vehicle metadata — Tata Signa 4825.TK specs in the hackathon demo.

Data flow into the database

FastAPI's MQTT listener writes to **sensor_readings** continuously. A scheduled job (or on-demand call) reads recent rows, computes features, and the ML service writes results to **predictions**. If failure probability crosses a threshold, an **alerts** row is created, referencing the matching row in **sop_documents** retrieved via RAG. The dashboard queries all four tables to render the live view.

7. IoT Simulation & Backend — Explained

Simulation tool: Wokwi (browser-based circuit simulator, no install). **Backend:** FastAPI service with a background MQTT listener, running locally on one team laptop during the hackathon.

IoT simulation (Wokwi)

An ESP32 board is simulated in Wokwi with virtual sensor components (temperature, vibration/accelerometer, analog pressure inputs) wired in the browser — no physical hardware needed for Phase 1. The simulated firmware (written in Arduino-style C++ or MicroPython) reads each virtual sensor on a loop, formats the reading as JSON, and publishes it to an MQTT broker over WiFi, exactly as a real ESP32 would. Failure scenarios are scripted directly into the simulation — e.g. a function that ramps vibration and temperature upward over a fixed time window — so the team can trigger a "failure" reliably on demand during the live demo instead of relying on random noise.

MQTT broker

A lightweight public or locally-run broker (e.g. Mosquitto, or a free hosted broker such as HiveMQ's public instance for the demo) sits between Wokwi and the backend. Topics are namespaced per truck and sensor, e.g. `edgeguard/truck1/temperature`, `edgeguard/truck1/vibration`, so the backend can subscribe to all topics under a truck and route by sensor type.

Backend (FastAPI)

- 1. MQTT listener:** runs as a background task inside the FastAPI app at startup, subscribing to all sensor topics and pushing incoming messages into an async queue.
- 2. Cleaning layer:** a worker reads the queue, discards malformed messages, clips obvious out-of-range spikes, and timestamps each reading.
- 3. Buffering:** if the database write fails or connectivity drops, readings are held in an in-memory or on-disk buffer and flushed once the connection recovers — this is what's being demonstrated, not just described, when judges ask about offline mining connectivity.
- 4. REST endpoints:** FastAPI exposes endpoints for the dashboard to pull recent readings, latest predictions, and active alerts, and an endpoint for the ML service to push new predictions back in.
- 5. Inference trigger:** on a schedule (e.g. every 30 seconds) or on new-data-arrival, the backend calls the ML model with the latest feature window and stores the result.

Why this is laptop-feasible in 5 days

Every component — Wokwi, Mosquitto/public broker, FastAPI, SQLite, XGBoost — runs on a single laptop with no cloud account, no paid service, and no physical hardware purchase required for the demo itself. This keeps Phase 1 entirely buildable within the hackathon's laptop-only constraint.

8. Problem Statement, Solution & Cross-Truck Feasibility

Problem	Our Solution
Unplanned breakdowns — bearings, hydraulics, suspension, engine, and battery fail without warning, costing ~Rs 2,00,000+ per incident (repair + 10hr downtime + production loss).	Edge-first sensor pipeline + XGBoost/LightGBM model predicts failure probability and Remaining Useful Life (RUL) hours ahead of time, so maintenance is scheduled, not reactive.
No early-warning signal exists today — operators only find out a component is failing when it already has.	Continuous sensor monitoring (temperature, vibration, oil pressure, hydraulic/suspension pressure) feeds rolling-window features into the model every cycle, catching the temperature→vibration→pressure degradation cascade as it starts, not after.
Maintenance crews don't know what to do when an alert fires — raw sensor numbers aren't actionable on their own.	RAG layer (Chroma/FAISS) retrieves the matching SOP/manual section and the copilot explains the alert in plain language with a recommended action, not just a number.
Connectivity is unreliable at mine sites — cloud-only systems lose data exactly when conditions are worst.	Edge-first design: data is cleaned and buffered locally at the truck/gateway level and synced when connectivity returns, instead of depending on a constant live link.
No historical failure data exists yet to train on.	Synthetic degradation-cascade data generation lets the model learn the failure pattern now; the same pipeline retrains seamlessly once real fleet data starts arriving in Phase 2/3.

Feasibility: can one build generalize across all truck models?

Short answer: **yes for the architecture, partially for the trained model** — and that distinction matters for the pitch.

What transfers directly, with zero rework: the entire pipeline downstream of the sensor — MQTT schema, FastAPI ingest/cleaning, database structure, RAG/SOP retrieval, and dashboard — is truck-agnostic by design. None of that code references the Signa 4825.TK specifically; it only expects a `sensor_type` and a value. Point it at a different truck's sensors and it runs unchanged.

What needs adjustment per truck class: the trained ML model itself. Failure thresholds depend on a truck's specific GVW, payload, hydraulic operating pressure range, and component tolerances — a 47.5t Signa 4825.TK and a smaller 19t tipper genuinely degrade differently under the same vibration reading. A model trained purely on Signa 4825.TK data will carry over reasonably well to other *similar-class* heavy tippers (same sensor types, same failure modes, similar pressure ranges), but accuracy drops the further you move from that vehicle class — e.g. applying it unmodified to a light commercial truck would be a stretch.

Realistic path to true fleet-wide generalization: keep the same sensor types and feature engineering, but make truck class (GVW, axle count, payload rating) an input feature to the model rather than a fixed assumption, and retrain on a small amount of data per truck class as it becomes available. This is a data and calibration problem, not an architecture rebuild — which is the credible claim to make to a mentor: *"the system generalizes across Tata's tipper range with per-class calibration, not a from-scratch rebuild per model."*

Hackathon-stage honesty: for this 5-day build, train and demo against the Signa 4825.TK only, but state explicitly in the pitch that the architecture is vehicle-agnostic and the model is the one component that needs lightweight per-class retraining — this reads as engineering maturity, not a limitation.

9. Full Workflow — IoT Simulation to Dashboard (No Physical Sensors)

Every step below runs entirely on team laptops. No physical sensor, microcontroller, or truck wiring is used anywhere in this flow — Wokwi's virtual ESP32 and virtual sensor components stand in for all hardware for the full Phase 1 build.

Step	Member	Role	Work (No Hardware)
1. Virtual Circuit Setup	Member 5	Sensor Handler	Builds the ESP32 board and virtual sensor components (temperature, accelerometer/vibration, analog pressure pins) entirely inside the Wokwi browser editor — no physical parts ordered or wired.
2. Simulated Firmware	Member 5 & 2	Sensor Handler + Full-Stack Dev	Writes Arduino-style C++ code that generates realistic virtual sensor values (including the temperature→vibration→pressure degradation cascade) and a scripted failure trigger for the live demo.
3. MQTT Publish	Member 2	Full-Stack Dev	Wokwi's simulated WiFi connects out to a real but lightweight MQTT broker (e.g. public HiveMQ or local Mosquitto); firmware publishes each virtual reading as JSON to a per-sensor topic.
4. MQTT Subscribe	Member 2	Full-Stack Dev	FastAPI app's background listener subscribes to all truck/sensor topics at startup and pushes incoming messages into an async queue for processing.
5. Clean & Buffer	Member 2	Full-Stack Dev	Worker validates message format, clips out-of-range spikes, timestamps each reading, and buffers locally if the DB write fails — simulating a connectivity drop on demand.
6. Store	Member 3	Database Handler	Cleaned readings are written to the sensor_readings table in SQLite with truck_id, sensor_type, value, and timestamp.
7. Feature Engineering	Member 3 & 4	DB Handler + ML Engineer	Member 3 queries recent rows; Member 4 computes rolling mean, rolling std, and rate-of-change features per sensor channel over the trailing window.
8. Model Inference	Member 4	ML Engineer	Trained XGBoost/LightGBM models (trained earlier on synthetic degradation data) run on the feature window, returning failure probability and RUL hours; result written to predictions table.
9. RAG Retrieval	Member 4	ML Engineer	If failure probability crosses threshold, Chroma/FAISS retrieves the matching SOP chunk for the flagged component; an alerts row is created referencing it.
10. API Exposure	Member 2	Full-Stack Dev	FastAPI REST endpoints serve the latest readings, predictions, and alerts on request, so the frontend never queries the database directly.
11. Dashboard Display	Member 1	Frontend Developer	Frontend polls/calls the REST endpoints, renders live telemetry charts, surfaces the alert with its RUL estimate, and shows the copilot's plain-language explanation.
12. Demo Orchestration	Member 1 & 5	Frontend Dev + Sensor Handler	Member 5 triggers the scripted failure in Wokwi on cue; Member 1 ensures the dashboard reflects it live in front of the mentor/judges within seconds.

Zero hardware checkpoint: Steps 1-2 replace physical sensors and microcontrollers with Wokwi's browser simulation; Steps 3-12 are identical to how the system would behave with real hardware, which is exactly why moving to Phase 2 later only swaps Steps 1-2 for a real ESP32 — nothing downstream needs to change.